Penetration Testing Report

Executive Summary

This report details the results of a penetration testing training exercise completed using PortSwigger Web Security Academy. The objective of this exercise was to identify and exploit common web application vulnerabilities in a safe, legal, and educational environment, then document the findings in a professional penetration testing report.

The testing covered 11 beginner-level web security labs across several common vulnerability categories, including SQL injection, cross-site scripting, access control, authentication weaknesses, path traversal, and OS command injection. These vulnerabilities represent common security issues that can affect real-world web applications if user input, authentication, authorization, and server-side functionality are not implemented securely.

The findings demonstrate how attackers may gain unauthorized access, view sensitive information, manipulate application behavior, or execute unintended actions when appropriate security controls are missing. Recommendations are provided for each finding to help reduce the likelihood and impact of similar vulnerabilities in real-world applications.

Overview of the Web Application Tested

PortSwigger Web Security Academy is an online learning platform that provides free, safe, and legal web security labs. The labs are designed to help learners understand how common web application vulnerabilities work and how they can be identified, exploited in a controlled environment, and prevented.

For this task, 11 beginner-level labs were completed. The purpose of the exercise was not only to complete the labs, but also to understand the security weaknesses demonstrated in each lab and explain how similar issues could be prevented in a real-world organization.

Scope of Testing

The scope of testing was limited to the PortSwigger Web Security Academy training labs selected for this task. Testing was conducted only within the legal training environment provided by PortSwigger.

The following types of vulnerabilities were included in scope:

- SQL injection
- Cross-site scripting
- Access control weaknesses
- Authentication vulnerabilities
- Path traversal

- OS command injection

No testing was conducted against real websites, third-party systems, or applications outside the PortSwigger Web Security Academy environment.

Key Findings

Lab	Vulnerability Description	Findings	Recommendation(s)
Lab 1: SQL Injection — Login Bypass	SQL injection in a login function	The login function did not properly validate or parameterize user input. As a result, the authentication logic could be manipulated to bypass normal login requirements.	Use parameterized queries or prepared statements for all database queries. Avoid building SQL queries directly from user input. Implement strong authentication controls and monitor for unusual login attempts.
Lab 2: SQL Injection — Hidden Data Exposure	SQL injection in a product/category filter	The application accepted user-controlled input in a database query, allowing hidden or unauthorized data to be retrieved.	Use prepared statements, server-side input validation, least-privilege database accounts, and error handling that does not reveal database structure.
Lab 3: Reflected Cross-Site Scripting	Unsanitized user input reflected in the page response	User input was reflected back into the webpage without proper encoding. This could allow an attacker to execute malicious JavaScript in another user's browser.	Encode output based on context, validate user input, and implement a strong Content Security Policy where appropriate.
Lab 4: Stored Cross-Site Scripting	Malicious input stored and displayed to other users	The application stored user-submitted content and displayed it without proper sanitization. This could allow malicious scripts to affect multiple users.	Sanitize and encode stored user content before rendering it. Use allowlists for expected input and apply secure content handling throughout the application.
Lab 5: DOM-Based Cross-Site Scripting	Client-side JavaScript handled user input insecurely	The vulnerability occurred in the browser when client-side JavaScript used user-controlled data in an unsafe way. This shows that security controls	Avoid writing untrusted data directly into the DOM. Use safe DOM APIs and validate or sanitize data before it is processed by client-side scripts.

Lab	Vulnerability Description	Findings	Recommendation(s)
		must also account for front-end code.	
Lab 6: Unprotected Admin Functionality	Administrative functionality was accessible without proper protection	The application exposed administrative functionality without requiring sufficient authorization. This could allow unauthorized users to perform privileged actions.	Enforce server-side access control checks for all administrative functions. Do not rely on hidden URLs or client-side controls to protect sensitive functionality.
Lab 7: Hidden Admin Functionality	Sensitive functionality was discoverable through application clues	Administrative functionality was not clearly linked in the interface but could still be discovered. This demonstrates that obscurity is not an effective access control method.	Protect all sensitive routes with authentication and authorization checks. Ensure users cannot access privileged functions unless their role explicitly permits it.
Lab 8: Username Enumeration	Authentication responses revealed valid usernames	The application responded differently depending on whether a username was valid. This could help attackers identify real accounts before attempting password attacks.	Use consistent error messages for failed login attempts. Apply rate limiting, account lockout protections, and monitoring for repeated authentication failures.
Lab 9: Two-Factor Authentication Bypass	Multi-factor authentication process could be bypassed	The authentication flow did not properly enforce the second factor before granting access. This could allow attackers to bypass an important security control.	Ensure multi-factor authentication is enforced server-side before a session is fully authenticated. Validate each step of the login process and prevent users from skipping required checks.
Lab 10: File Path Traversal	User input allowed access to unintended files	The application used user-controlled input to retrieve files, allowing access outside the intended directory. In a real application, this could expose sensitive files.	Validate file paths, restrict file access to approved directories, avoid passing raw user input into file system operations, and use allowlists for permitted files.

Lab	Vulnerability Description	Findings	Recommendation(s)
Lab 11: OS Command Injection	User input was passed into a system command	The application passed user-controlled input into an operating system command without sufficient validation. This could allow an attacker to execute unauthorized commands on the server.	Avoid using OS commands when safer application-level alternatives exist. If commands are required, strictly validate inputs, use allowlists, and separate user input from command execution.

Overall Analysis

The completed labs demonstrate several important themes in web application security. First, user input should never be trusted by default. Many of the vulnerabilities occurred because the application accepted user input and processed it without proper validation, sanitization, encoding, or parameterization.

Second, authentication and authorization must be enforced on the server side. Hidden pages, unlinked functionality, or client-side checks are not sufficient protections. If a user can directly access a URL or manipulate a request, the server must still verify whether that user is allowed to perform the action.

Third, secure coding practices must be applied across both the front end and back end. Vulnerabilities such as DOM-based cross-site scripting show that browser-side logic can introduce risks, while SQL injection, path traversal, and command injection demonstrate the importance of secure server-side handling.

Finally, penetration testing reports are important because they help organizations understand not only what vulnerabilities exist, but also why they matter and how they can be remediated. A strong report should communicate risk clearly to both technical and non-technical audiences.

Conclusion

The penetration testing exercise identified a range of common web application vulnerabilities, including SQL injection, cross-site scripting, access control weaknesses, authentication flaws, path traversal, and OS command injection. If these vulnerabilities existed in a real-world application, they could potentially lead to unauthorized access, data exposure, account compromise, or server-side compromise.

The most important recommendations are to validate and sanitize user input, use parameterized queries, encode output, enforce server-side access controls, strengthen authentication flows, and avoid unsafe handling of file paths or operating system commands.

Regular penetration testing, secure code reviews, and developer training can help organizations identify and fix these issues before they are exploited by attackers. By completing this exercise, learners gain practical experience in recognizing common vulnerabilities and communicating findings in a structured, professional report.

Appendix

Students should include evidence of completion in this section, such as screenshots of completed PortSwigger Web Security Academy labs.

Suggested appendix format:

- Appendix A: SQL Injection — Login Bypass completion screenshot
- Appendix B: SQL Injection — Hidden Data Exposure completion screenshot
- Appendix C: Reflected XSS completion screenshot
- Appendix D: Stored XSS completion screenshot
- Appendix E: DOM XSS completion screenshot
- Appendix F: Unprotected Admin Functionality completion screenshot
- Appendix G: Hidden Admin Functionality completion screenshot
- Appendix H: Username Enumeration completion screenshot
- Appendix I: Two-Factor Authentication Bypass completion screenshot
- Appendix J: Path Traversal completion screenshot
- Appendix K: OS Command Injection completion screenshot